

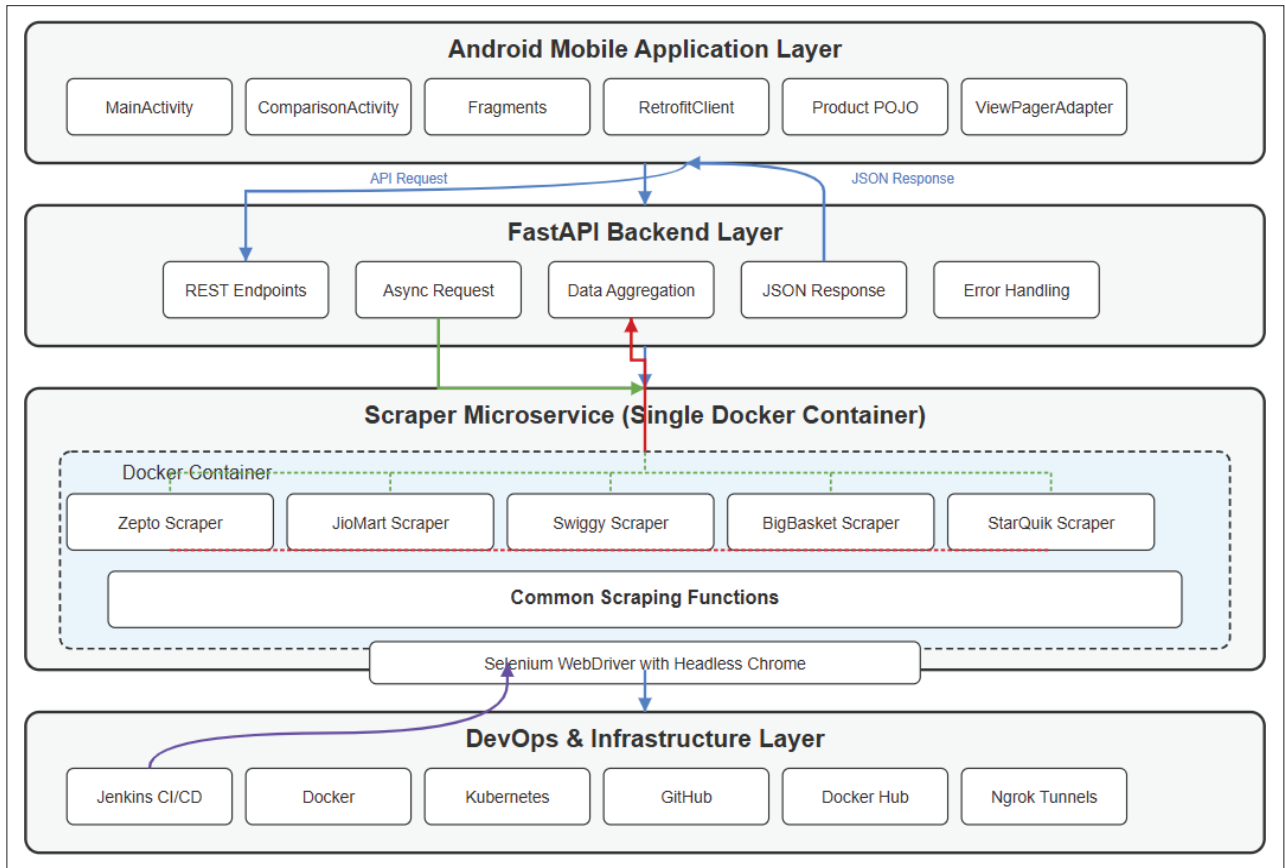


Figure 1: Microservices-based system architecture depicting data flow for real-time grocery price comparison

Scrapers extract structured data including product name, price, unit, availability, and image URL, either by parsing rendered DOM elements or intercepting API calls using Chrome Dev-Tools Protocol.

FastAPI backend server: The FastAPI backend exposes RESTful endpoints to deliver scraped data to external consumers, particularly the Android mobile frontend. It is architected to support asynchronous requests and rapid response times. The API exposes end points such as `/search?query={product}`, which return JSON formatted data. The server also handles request routing, response formatting, and basic input validation.

Mobile application layer: An Android-based mobile application built using Java serves as the user interface layer. It interacts with the FastAPI backend via HTTP APIs, rendering

product listings and comparison data in a structured and user-friendly format. The app is designed for minimal data loading times and efficient rendering of image and price data.

CI/CD pipeline with Jenkins:

A Jenkins-based CI/CD pipeline automates building, testing, and deploying all components. For the mobile application, Jenkins clones the GitHub repository, builds the APK using Gradle, and archives the artifact. For backend and scrapers, Jenkins builds Docker images and publishes them to Docker Hub. Deployment is performed using Kubernetes manifests via the Jenkins Kubernetes plugin and `kubectl` commands.

Containerisation and

orchestration: All backend and scraper services are containerised using Docker and deployed to a Kubernetes cluster. Kubernetes is used to manage service

discovery, load balancing, rolling updates, and fault tolerance. This setup ensures that each microservice can scale independently based on resource usage and demand.

Experimental setup

To validate the system functionality and performance, a structured experimental setup was designed, simulating near production conditions. The objective was to test real-time data retrieval, API responsiveness, Android integration, and deployment reliability across a modular microservices environment.

Environment configuration: The experimental environment consisted of local development machines running Linux and Windows 11, with the scraper and backend services containerised using Docker. The Android application was tested on physical devices (Samsung Galaxy S23, One Plus)